

SemWeaver: Refining LLM-Generated Static Vulnerability Checkers with Analyzer-Internal Evidence

ANONYMOUS AUTHOR(S)

LLM-generated static vulnerability checkers often warn on both vulnerable and fixed code because a patch alone underspecifies guards, state, and data flow. We study post-synthesis refinement using analyzer-internal representations: backend-produced records bound to raw output and the source revision, file, and function, distinct from diagnostics and source-window fallbacks. SEMWEAVER organizes these records with patch, source, and validation context, accepting an edit only if it retains a vulnerable-side hit while silencing the fixed side. Among 39 materialized KNighter-derived CSA checkers, 12 are fixed-noisy; their initial 0/12 patch-discriminating success (PDS) is a selection criterion, not KNighter's refinement performance. Across three full-feedback repeats with matched per-case call ceilings on these 12 Linux cases, KNighter's actual loop achieves 3, 2, and 6 PDS; native-evidence SEMWEAVER achieves 6, 5, and 7, while its no-internal arm achieves 3 each time. Under PDS-only adoption, native SEMWEAVER retains 23, 23, and 18 of 37 starting fixed-side alerts versus KNighter's 28, 27, and 22. Of 70 evidence records, 37 meet the strict internal definition. These development-informed repeated results indicate bounded patch-local false-positive reduction; no individual paired difference is significant, and no unseen-code generalization is claimed.

ACM Reference Format:

Anonymous Author(s). 2027. SEMWEAVER: Refining LLM-Generated Static Vulnerability Checkers with Analyzer-Internal Evidence. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Static analysis is a cornerstone of software security assurance for systems code. Tools such as CodeQL [17] and Clang Static Analyzer (CSA) [32] detect vulnerabilities by checking source code against predefined rules without executing it [4, 7, 14, 42]. The effectiveness of these tools depends on the quality of their vulnerability checkers, each of which must encode the semantic conditions (path guards, state invariants, data-flow relations, and API contracts) that distinguish a real bug from safe code. Writing such checkers by hand requires deep expertise in both the target codebase and the analysis framework, making manual authoring expensive, slow, and difficult to scale.

LLM-based patch-driven synthesis offers a promising alternative. In this paradigm, a security patch serves as a compact before/after specification: the LLM consumes the patch and synthesizes a reusable checker that should fire on the vulnerable revision but remain silent on the fixed revision. KNighter [48], a representative system, has demonstrated that this approach can produce compilable and executable CSA checkers from historical Linux kernel fixes.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE 2027, July 12–16, 2027, Shenzhen, China

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

However, the resulting checkers may be semantically inadequate. A security patch is inherently underspecified: a one-line `goto`-target change may encode an ownership barrier, a widened cast may encode a range guard, and a moved `free` may encode a lifetime invariant. A checker that captures only the syntactic shape of the edit, without encoding the underlying semantic condition, can fire on both the vulnerable and fixed revisions. After naïve refinement, the checker may instead lose the vulnerable-side trigger entirely, a failure mode we call over-pruning. In both cases the root cause is the same: the checker learned the syntactic edit but not the semantic condition it enforces.

Current refinement strategies, including KNighter’s own iterative loop, attempt to address this problem by operating at the output level [35, 44, 48]. They run the checker on both revisions, observe whether it fires on the wrong side, and feed this behavioral signal back to the LLM. Such feedback reveals the symptom (“the fixed side still warns”) but does not diagnose the cause: it cannot tell the LLM whether the missing predicate is a path guard, a cleanup barrier, an ownership transition, or an API contract. The LLM must therefore guess the missing semantic condition, reducing refinement to prompt-level retry.

We make a narrower observation: **some patch-relevant program structure is available in analyzer-produced intermediate representations** but is not ordinarily surfaced to the refinement loop. Path-sensitive analyzers such as CSA internally track branch guards, symbolic states, memory regions, and lifetime events during checker execution [32]. Relational query engines such as CodeQL maintain databases of data-flow edges, call-graph relations, type hierarchies, and API-usage patterns [2, 17]. These representations can suggest candidate predicates, but a CFG edge or call relation is not by itself a proof of path feasibility or checker correctness.

Exploiting these intermediate results, however, is not straightforward. A single CSA run can produce hundreds of path-sensitive states, and a CodeQL database exposes thousands of relational tuples. Feeding all of them into the refinement prompt overwhelms the context window, dilutes the relevant facts with noise, and makes it harder for the model to identify the specific predicate that needs to be added. The challenge is to select and present only the evidence that is relevant to the particular semantic gap in the checker under refinement.

We present SEMWEAVER, a post-synthesis checker editor guided by provenance-labelled analyzer context. SEMWEAVER is designed as a plug-in post-generation layer that is independent of the upstream LLM generator, treating any generated checker (from KNighter or another pipeline) as an artifact to be refined rather than regenerated. The framework addresses the evidence-selection challenge through three mechanisms. First, a rule-guided preflight extracts patch facts (e.g., guards, type widening, and affected calls) and prioritizes evidence requirements by inferred mechanism. The editor may request role views in the general workflow; the strict CSA comparison instead freezes its patch-scoped view before editing. For instance, a cleanup-edge patch triggers collection of path-guard and lifetime evidence from CSA, while a bounds patch triggers guard, flow, and API-usage evidence from CodeQL. Second, backend outputs, patch facts, source context, and validation feedback are normalized into a compact bundle with typed records, giving the editor a structured edit target (“add the patch-introduced cleanup barrier while preserving the freed-object transition”) instead of a vague behavioral symptom (“fixed side still warns”). Third, a strict target-preserving gate accepts a refinement only when the refined checker both preserves the vulnerable-side trigger and eliminates the fixed-side warnings, preventing over-pruning by construction.

We evaluate SEMWEAVER across three settings. We screen all 39 materialized KNighter-derived Linux CSA checkers: 12 are fixed-noisy and form the refinement stratum, while

already-discriminating checkers remain in the portfolio denominator. The selected stratum’s initial 0/12 is definitional, not a KNighter refinement baseline. A historical follow-up has five automatic PDS edits, six manual completions, and one loss, but predates strict analyzer-internal evidence. In three later, matched full-feedback repeats, the actual KNighter loop reaches 3, 2, and 6 PDS on the 12 cases; provenance-gated SEMWEAVER reaches 6, 5, and 7, retaining fewer fixed-side alerts in each repeat under PDS-only adoption. The no-internal arm reaches 3 each time. These are descriptive results on the same selected cases, not significant per-repeat superiority or unseen-code generalization; historical cross-backend and one-run model results remain exploratory.

This paper makes the following contributions:

- We identify the gap between output-level behavioral feedback and analyzer-internal semantic evidence for checker refinement, and formulate the problem of target-preserving semantic refinement for LLM-generated vulnerability checkers.
- We present SEMWEAVER, a plug-in refinement layer that organizes provenance-labelled analyzer representations, analyzer outputs, patch and source context, and validation feedback into a compact semantic evidence bundle; the strict CSA comparison freezes the available view rather than evaluating adaptive routing.
- We provide a denominator-complete audit of KNighter-derived Linux checkers and separate historical, matched-budget, cross-backend, ablation, and failure-analysis results by their provenance and inferential scope.

2 BACKGROUND

This section introduces the background on static vulnerability checkers, LLM-based checker generation from security patches, and the analyzer-internal intermediate results that motivate our approach.

2.1 Static Vulnerability Checkers

Static analysis examines source code without executing it, aiming to find bugs, security vulnerabilities, and code-quality issues at compile time or during code review [4, 14]. A vulnerability checker is a rule or query within a static-analysis framework that flags program locations matching a security-relevant condition. Modern frameworks support different checker paradigms. Path-sensitive engines such as the Clang Static Analyzer (CSA) perform symbolic exploration over program paths, tracking branch guards, symbolic values, memory regions, and lifetime transitions to detect bugs like use-after-free, null dereference, and resource leaks [32]. Relational query engines such as CodeQL compile source code into a queryable database and express checkers as declarative queries over data-flow edges, call-graph relations, type hierarchies, and API-usage patterns [2, 17]. Both paradigms have been deployed at industrial scale for continuous vulnerability detection [7, 42].

Writing effective checkers remains a bottleneck. A checker must encode not only the syntactic pattern of a vulnerability but also the semantic conditions (path guards, state invariants, data-flow relations, and API contracts) that distinguish real bugs from safe code. This requires deep domain knowledge of both the target codebase and the analysis framework, making manual checker authoring expensive and error-prone.

2.2 LLM-Based Patch-Driven Checker Generation

Recent work has proposed using LLMs to automate checker generation from security patches. A security patch provides a compact before/after specification: the vulnerable revision

should trigger the generated checker, and the fixed revision should not. This idea builds on a well-established line of work that treats patches as vulnerability specifications, including semantic patching [38], clone-based vulnerability discovery [19, 24], and patch-derived datasets [15]. KNighter [48] is a representative system that synthesizes CSA checkers from historical Linux kernel fixes and validates them against the vulnerable and fixed revisions.

However, the generated checkers are seldom semantically adequate. A patch is inherently underspecified: a one-line edit may encode a path guard, an ownership transition, a lifetime barrier, or an API contract, yet the generated checker often captures only the syntactic shape of the edit without the semantic condition the edit enforces. Existing refinement loops, including KNighter’s own iterative strategy, operate at the output level [35, 44, 48]. They feed behavioral signals (e.g., “the fixed side still warns”) back to the LLM, revealing the symptom but not diagnosing the cause. This feedback cannot tell the LLM whether the missing predicate is a path guard, a cleanup barrier, an ownership transition, or an API contract, so the model must guess which semantic condition to add.

2.3 Analyzer-Internal Intermediate Results

Static-analysis backends compute intermediate representations that can be exposed through debugging interfaces or database queries, but these are not normally supplied to a checker-refinement loop.

CSA, as a path-sensitive engine, internally maintains per-path symbolic states that record branch conditions, memory region bindings, allocation/free events, and object lifetime transitions [32]. CodeQL, as a relational engine, materializes a full program database from which data-flow edges, call-graph relations, type hierarchies, member-access patterns, and API-usage sequences can be queried [2, 17]. These intermediate facts have long underpinned taint analysis, graph-based detection, and interprocedural security reasoning [1, 21, 31, 46].

These representations may suggest a missing predicate, but a CFG branch or database tuple does not itself establish vulnerability semantics. The operational distinction between a backend-produced representation, a diagnostic, and a source-window fallback is central to our evidence accounting (Section 4.2). This motivates extracting bounded, provenance-labelled context for refinement while retaining paired validation as the correctness gate.

3 MOTIVATING EXAMPLE

Consider the KNighter-derived CSA checker for Linux commit 5aa2184e2908, which fixes a double-free error path in `mlx5hws_definer_calc_layout` [25, 48]. Figure 1 shows two fallible calls: conversion helper `hws_definer_conv_match_params_to_h1` (❶) establishes ownership of `mt->fc` only on success; a later layout-fitting failure may then legitimately release it through `free_fc`. On conversion failure, however, the vulnerable `goto` targets `free_fc` (❷) before ownership exists. The patch redirects only that edge to `free_match_h1` (❸), which frees the already-owned `match_h1` while bypassing `mt->fc`. The needed predicate distinguishes these guarded error paths, not merely the presence of a free call.

Checker behavior and evidence. The starting checker (❹) tracks `kzalloc` regions and warns at `kfree` when a region is absent from its map. It misses the guarded ownership distinction and emits 10/9 patch-local alerts on vulnerable/fixed revisions. A fixed-side report shows the symptom but does not identify the conversion call, guard, cleanup edge, and owned object that form the separator. The evidence bundle places the two cleanup labels and guarded edges in one semantic slice; a Clang CFG dump supports branch context, while the ownership interpretation comes from patch and source analysis, *not* a path-feasible

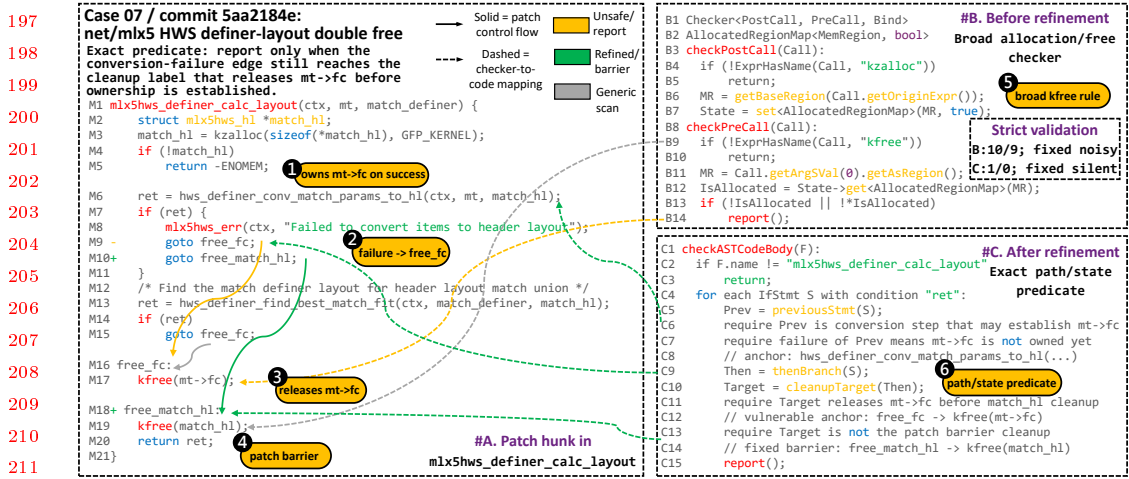


Fig. 1. **Motivating double-free refinement.** Solid arrows show patch control flow. Dashed arrows show checker-to-code mappings. Orange highlights the unsafe path, teal the patched barrier, and gray the legitimate later flow. The baseline checker (5) fires broadly (10/9). The manually completed checker (6) encodes the conversion-failure ownership predicate and achieves 1/0; this case is not an automatic-refinement success.

analyzer proof. A validation-only edit can simply suppress both sides: the historical one-run ablation here produced 0/0.

The checker in Figure 1c was *manually completed*: its AST-body predicate recognizes the conversion-failure edge and changes paired counts to 1/0. It is not an automatic SEMWEAVER success. Modeling the allocator or tracked-state check in the original symbolic checker would be another plausible repair; this example does not show that the AST rewrite is preferable. The shown predicate names the patch function and is therefore patch-bound, not a general double-free detector. The strict PDS gate establishes only the paired result, not stability under future changes to the code or callback names.

4 APPROACH

SEMWEAVER is a post-synthesis refinement layer for LLM-generated vulnerability detectors. Given a draft CSA checker or CodeQL query produced by an upstream generator, the patch that motivated the synthesis, and the corresponding source tree and validation scope, SEMWEAVER refines the detector to encode missing semantic predicates while preserving the vulnerable-side trigger.

Figure 2 shows five stages. **(a) Generator context.** SEMWEAVER receives a draft detector from an upstream LLM generator (e.g., KNighter), together with the patch, source tree, and validation target. It treats the generator as a black box: any synthesized detector is an artifact to be refined, not regenerated. **(b) Evidence assembly.** Rule-guided patch preflight infers a likely mechanism from patch facts and available analysis metadata, then prioritizes evidence requirements. Four collectors (patch preflight, source extraction, analyzer representations or outputs from CSA/CodeQL, and validation feedback) gather evidence (Section 4.1). **(c) Evidence bundle.** All facts are normalized into a typed `EvidenceBundle` with seven persisted record types, giving the refinement agent structured edit targets rather than raw analyzer dumps (Sections 4.2 and 4.3). **(d) Target-preserving refinement.**

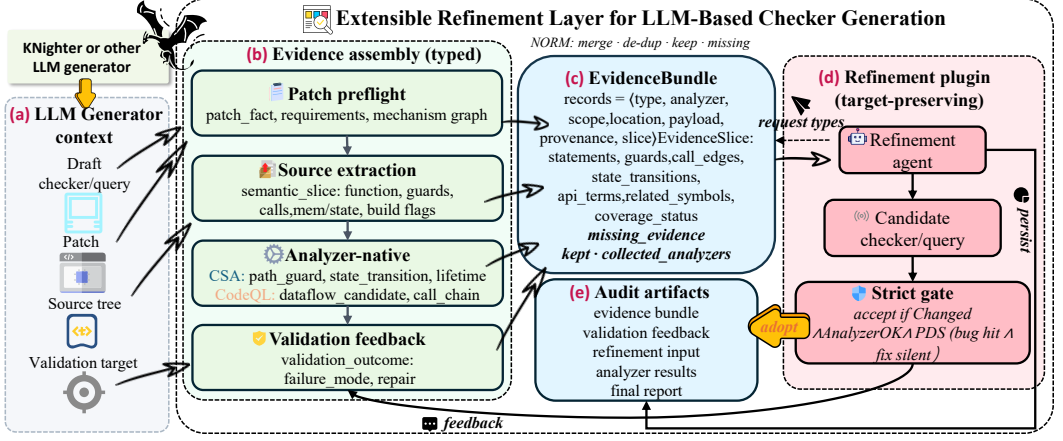


Fig. 2. **SemWeaver architecture.** (a) An LLM generator produces a draft detector from the patch. (b) A rule-guided patch preflight plans evidence requirements and four collectors gather corresponding facts; the figure’s “analyzer-native” collector label does not mean every emitted record has internal provenance. (c) Facts are normalized into a typed **EvidenceBundle** with seven persisted record types. (d) The refinement agent edits the detector using role-specific views. A strict gate accepts only candidates that preserve the vulnerable-side trigger while silencing the fixed side. Rejected candidates feed back into the bundle (dashed arrow). (e) Adopted candidates, evidence bundles, and validation logs are persisted as audit artifacts for reproducibility.

The refinement agent edits the detector using role-specific evidence views from the bundle. A strict gate accepts a candidate only when it preserves the vulnerable-side trigger and eliminates fixed-side warnings. Rejected candidates re-enter the bundle as feedback for the next iteration (Section 4.4). (e) **Audit artifacts.** Each run persists the starting and candidate detector, evidence bundle, prompt/response trace, validation logs, and final report. The verifier recomputes raw-output hashes, checks the recorded Clang interface/schema and patch file/function binding for every internal record, recounts provenance classes, and then checks the compiled candidate against paired vulnerable/fixed logs. It does not infer evidence origin from an LLM explanation or rerun a model to reproduce a deterministic count.

4.1 Adaptive Evidence Selection

The central design choice in evidence assembly (Figure 2b) is *what to collect and expose*. Analyzer outputs can be large and heterogeneous; passing them wholesale to the editor obscures patch-relevant relations. The preflight planner represents candidate requirements by role and priority, while source/function scope and bounded views limit what the editor sees. In the strict CSA comparison, collection is precomputed and the visible view is frozen, so its result does not measure demand-driven routing.

Rule-guided preflight and role requests. The implementation has no separately trained or prompted patch-mechanism classifier. **PatchFactsExtractor** parses the patch and metadata for affected functions, added guards, risky calls, and type widenings. **Evidence Planner** takes a primary pattern from analysis metadata when available or infers it from those facts, then assigns fixed priorities to requirements: patch anchoring is 100, lifetime/state requirements for double-free or use-after-free are 92/89, and a guard for null or bounds

Algorithm 1 Target-Preserving Evidence-Guided Refinement

Require: detector D , patch P , source S , scope V , backends A , budget K
Ensure: refined detector D^* and audit report

```

1:  $R_0 \leftarrow \text{VALIDATE}(D, P, V)$ 
2:  $E_p, E_s \leftarrow \text{PREFLIGHT}(P), \text{EXTRACTSOURCE}(P, S, D)$ 
3:  $E_a \leftarrow \bigcup_{a \in A} \text{COLLECTNATIVE}(a, D, P, S)$     $E_v \leftarrow \text{FEEDBACK}(R_0)$ 
4:  $B \leftarrow \text{NORMALIZEMERGE}(E_p, E_s, E_a, E_v)$ 
5: if  $\text{PDS}(D, P, V)$  then
6:   return  $\text{AUDIT}(D, B)$ 
7: end if
8:  $D^* \leftarrow D$ 
9: for  $i = 1$  to  $K$  do
10:   $V_i \leftarrow \text{AGENTSELECTVIEW}(B)$     $C_i \leftarrow \text{AGENTEDIT}(D^*, P, V_i)$ 
11:   $R_i \leftarrow \text{VALIDATE}(C_i, P, V)$ 
12:  if  $\text{CHANGED}(C_i, D^*) \wedge \text{ANALYZEROK}(R_i) \wedge \text{PDS}(C_i, P, V)$  then
13:     $D^* \leftarrow C_i$    break
14:  else
15:     $E_i \leftarrow \text{FEEDBACK}(R_i)$     $B \leftarrow \text{MERGE}(B, E_i)$ 
16:  end if
17: end for
18: return  $\text{AUDIT}(D^*, B)$ 
```

Table 1. Rule-guided evidence requirements by patch mechanism (representative); names are persisted record types, not gold labels or measured classifier outputs.

Patch mechanism	Possible record types	Backend
Cleanup edge / double free	allocation_lifecycle, transition, path_guard	state_ CSA
Bounds widening / overflow	semantic_slice, dataflow_candidate, state_transition	CSA/CodeQL
Initialization / null deref	state_transition, path_guard	CSA
Use-after-free / escape	allocation_lifecycle, call_chain	CSA/CodeQL
API misuse / missing check	semantic_slice, call_chain	CodeQL

cases is 90. It sorts by descending priority with evidence type as a tie-breaker and retains at most eight planned requirements in the default configuration. Table 1 summarizes mechanism-to-role examples; it is not a classifier confusion matrix or a promise that every role is analyzer-internal. The normalizer does not learn or re-rank facts. In the general workflow the editor may request evidence types, with duplicate and request-count limits. For the strict provenance-audited CSA comparison, patch-scoped records are precomputed. The treatment preloads up to the first four distinct strictly internal record types in persisted order and disables additional requests; its no-internal counterpart sees no internal records but retains patch, source, and feedback context. This deterministic exposure policy is a limitation, not an adaptive classifier. The matched result tests the frozen native-evidence view, *not* a routing policy’s accuracy or benefit.

4.2 Evidence Sources and Roles

Given the patch scope and any planned requirements, four collectors execute in sequence (Figure 2b, top to bottom):

Table 2. Evidence primitives and provenance in the audited 12-case CSA cohort. One record may contain several CFG primitives; counts are by record origin, not primitive.

Origin	What the persisted record contains	Records
Analyzer-internal	Clang CFG branches/guards, value bindings, and call-graph edges, bound to raw dump and function	37
Analyzer output	CSA diagnostics and paired validation reports; no internal state	18
Source-derived	Patch facts, source windows, or compile-database-assisted slices	15

Patch preflight identifies changed files, hunks, and statements, and produces a mechanism graph that links the edited code to likely bug patterns. This supplies the structural skeleton: *what changed* and *where*.

Source extraction resolves patch files and records bounded contexts around the changed code, including enclosing functions, branch guards, callee signatures, memory/state operations, and build flags. This provides local semantic context without dumping repository-scale source.

Analyzer evidence is classified by its actual origin, not by the collector’s name. We call a record *analyzer-internal* only if its payload is derived from a backend-produced intermediate representation, identifies the interface and raw output, and is bound to a source revision, file, and function. In the measured CSA cohort the interface is Clang 18’s `debug.DumpCFG` and `debug.DumpCallGraph`; the persisted UTF-8 dump is parsed into schema `semweaver.csa_cfg_snapshot.v1`, with a raw-output SHA-256 and source-scope binding. These dumps provide CFG branches and value bindings and call edges. They do *not* themselves prove a path-feasible lifetime or ownership transition. Analyzer diagnostics and alert counts are *analyzer output*, not internal state. Patch diffs, source windows, and compile-database-assisted source extraction are *source-derived* fallbacks even when a CSA or CodeQL collector emits them. A live CodeQL database query could meet the internal-evidence definition if its query, database revision, and tuples were retained; we do not attribute source-window-derived CodeQL facts to that category.

This distinction makes extraction challenging in a practical sense: backend intermediates are verbose, backend-specific, sometimes unavailable for the patch-related translation unit, and must be linked to the same revision and function as the checker failure. The collector retains missing-evidence markers rather than silently upgrading fallback source text to internal evidence. Across the 12 CSA evidence bundles used in the provenance audit, 55 of 70 records are analyzer-produced: 37 meet the strict internal definition and 18 are analyzer outputs; the remaining 15 are source-derived. All 37 internal records have raw-dump and patch-scope bindings in the audit. These counts characterize evidence availability, not refinement success or CodeQL-native coverage.

Validation feedback records whether the baseline detector compiles, executes, hits the vulnerable target, remains fixed-side noisy, or loses the target entirely. This provides the failure *symptom*, while the preceding three sources supply the semantic *edit target*.

The persisted type vocabulary is `patch_fact`, `semantic_slice`, `dataflow_candidate`, `call_chain`, `path_guard`, `allocation_lifecycle`, and `state_transition`. A `patch_fact` anchors changed statements; `semantic_slice` summarizes bounded statements, guards, API terms, and related symbols. `dataflow_candidate` and `call_chain` describe possible propagation and call context, while `path_guard`, `allocation_lifecycle`, and `state_`

transition describe control, resource, and abstract-state context. “Barrier,” “type filter,” and “API usage” are semantic interpretations inside these records, not additional persisted types or claims of proven path feasibility.

When a selected role yields no evidence, the bundle retains an explicit missing-evidence marker rather than treating the absence silently, so the refinement agent knows what was requested but unavailable.

4.3 Evidence Bundle Schema

The normalizer merges all collected facts into the `EvidenceBundle` shown in Figure 2c. Each fact becomes an `EvidenceRecord` carrying a type, the contributing analyzer, scope, source location, payload, provenance, and an optional `EvidenceSlice` (the compact semantic content consumed by the agent). The schema separates program facts from analysis rules in the spirit of relational analyzer frameworks [2, 5, 20], but uses a smaller role vocabulary tailored to checker refinement. Formally:

$$\begin{aligned} B &= \text{Norm}(E_p \cup E_s \cup E_a \cup E_v), \\ E_a &= E_{\text{CSA}} \cup E_{\text{CodeQL}}, \\ r \in B.\text{records} &= \langle \text{type}, \text{analyzer}, \text{scope}, \\ &\quad \text{location}, \text{payload}, \text{provenance}, \text{slice} \rangle. \end{aligned} \tag{1}$$

Here E_p contains patch facts (changed statements, inferred bug mechanism), E_s contains bounded source slices around the patch and detector trigger, E_a contains analyzer-internal representations *or* analyzer outputs filtered by selected roles (Section 4.1), and E_v contains validation outcomes. Each record’s `provenance.origin` distinguishes `analyzer_internal`, `analyzer_output`, and `source_derived`. The normalizer deduplicates records and retains explicit markers for missing evidence; a record’s origin is not a confidence score or a proof of semantic correctness.

The type-to-backend mapping reflects available interfaces: CSA CFG/call-graph dumps inform guard, state, and call-chain records; CodeQL queries can support flow and relation records when a live database is available. These are record types, not seven distinct backend analyses. Cross-function call edges provide context, but the present CSA dump does not establish an interprocedural path or ownership proof. Provenance remains visible when a type is populated only by a fallback.

The bundle is bounded by design: preflight requirements and collectors restrict scope to patch-related files and validation targets, while the editor receives selected or preloaded role views rather than the full raw dump (Figure 2d). These constraints control prompt size and allow the contribution of strictly internal records to be tested by a matched ablation. A small, single-run ablation cannot by itself establish their causal contribution across the cohort.

4.4 Target-Preserving Refinement

The refinement loop (Figure 2d) receives the current detector, patch, baseline validation notes, and a bounded role-specific evidence view. The interface supports requesting further roles from the persisted bundle (“request types” arrow), but the provenance-audited matched condition freezes the visible view before the edit and disables additional agent requests. Thus its measured effect is a contrast between preselected native-evidence and no-internal views, not a test of dynamic evidence retrieval. This keeps the prompt bounded and the two conditions auditable.

The agent’s tool-mediated loop draws on recent LLM agent and self-refinement architectures but constrains actions to attached evidence, detector edits, and validation commands [35, 43, 44, 50]. Where evidence requests are enabled, they replay attached records rather than launch open-ended repository exploration; the request count is capped and unused or missing evidence is logged. In the matched condition, request count is zero by protocol.

Model versus deterministic components. LLMs may produce patch-analysis metadata and the editor’s checker edit; when enabled, the editor also chooses which evidence types to request. The preflight requirement rules, raw-output hashing and scope binding, record normalization, checker compilation, paired replay, and acceptance are programmatic. The LLM does not create a backend dump, set a provenance origin, or decide PDS. The audit retains the exact starting checker, prompt and response, visible roles, raw evidence, candidate source, and validation decision. This boundary prevents an LLM explanation from being counted as analyzer-produced evidence. The editor prompt supplies the current and starting checker paths, patch path, bounded source/evidence view, baseline vulnerable/fixed symptom, backend identity, and edit/validation rules. It asks for an in-place checker edit that models the mechanism rather than matching patch spellings, forbids suppressing the vulnerable trigger, and requires compilation before paired acceptance. Compiler-repair prompts receive the actual build diagnostics; all available English templates and rendered exchanges are named in the artifact manifest, with absent historical raw exchanges explicitly marked rather than reconstructed.

Strict gate. SEMWEAVER treats validation as both an analyzer-facing and a semantic gate (“Strict gate” in Figure 2d). A candidate must satisfy three conditions: (1) it has changed relative to the previous detector, (2) it compiles and executes under its backend harness (ANALYZEROK), and (3) it achieves patch-discriminating success (PDS), i.e., it reports the target on the vulnerable revision while remaining silent on the fixed revision. If a candidate becomes fixed-side silent by losing the vulnerable-side report, SEMWEAVER records a validity loss rather than a success, preventing over-pruning.

Feedback loop. When the gate rejects a candidate, the validation outcome is fed back into the bundle as new evidence (feedback arrow in Figure 2), enriching subsequent refinement attempts with the specific failure mode (e.g., “lost vulnerable-side trigger at line X”). The loop terminates when a candidate passes the gate or the budget K is exhausted (Algorithm 1). The repeated call-matched profile used below freezes the evidence view and submits at most one model edit before each external vulnerable/fixed replay. If the candidate fails the local build or the paired gate, the next permitted call receives hash-bound compiler or paired-alert feedback; the first strict PDS candidate stops the case. Its per-case ceiling is set by the actual KNIGHTER model calls in the same repeat. An earlier 19-call diagnostic performed paired replay only after the editor session and cannot measure this iterative-feedback profile; we do not pool its result with the repeats.

The concrete backend depends on the detector type: CSA checkers are compiled and replayed against both revisions, while CodeQL queries run over the pre-built database. Adding a backend requires an extractor that persists raw output and provenance fields, a mapping to editor-facing roles, and a paired validator that supplies the same H_v/S_f decision. Adding a bug mechanism requires a routing example, relevant role mapping, and paired positive/negative fixtures; it does not automatically require a new analyzer primitive. We have not measured engineer-hours for either extension and do not claim zero integration cost.

5 EVALUATION

We evaluate SEMWEAVER’s patch-related false-positive reduction and its sensitivity to verified analyzer-internal records. The scope is paired patch revisions and related build objects, not whole-kernel bug discovery.

5.1 Research Questions

RQ1: KNighter-derived checkers. Can SEMWEAVER improve strict patch discrimination for external CSA checkers derived from KNighter artifacts?

RQ2: Cross-backend feasibility and provenance. Which detector formats can the layer process, and what can the historical independent-generator data support once manual construction and missing lineage are accounted for?

RQ3: Evidence sensitivity. What changes when strict analyzer-internal records are withheld under an otherwise matched feedback protocol?

5.2 Subjects

We distinguish E1 (KNighter-derived Linux CSA checkers), E2 (historical cross-backend feasibility), and E3 (evidence sensitivity). The old five-case, single-run model study is retained in the artifact but excluded from effectiveness claims.

We start from the frozen, materialized 39-checker KNighter evaluation cohort, not only cases selected for refinement; this is *not* the universe of all checker candidates. After correcting a failed-analyzer validation for memory-leak commit 27834971, 27 checkers already satisfy PDS and 12 have a vulnerable-side hit but remain fixed-noisy. The 12 are the complete observed fixed-noisy stratum *within this cohort*, not a random or held-out sample. Their baseline PDS of 0/12 is definitional; it is *not* KNighter’s refinement performance. We retain the 39-checker denominator for portfolio-level reporting, and separately report the fixed-noisy stratum to measure how often the post-synthesis layer can remove its observed warnings. The frozen 39 include six local-triage, 17 positive-scan-selected, and 16 valid-but-no-local-report candidates across ten bug families; they are not a random sample. In a separate inventory of 286 generated artifacts, 45 carried an older `score_valid` flag, 35 overlapping the 39. All ten non-overlapping final-checker files were empty, so we explicitly substituted hash-bound repaired files and replayed the same paired object scope. Five already meet PDS, two are fixed-noisy, and three miss the vulnerable target; all ten executed. The resulting 49-commit union is a screening extension, *not* the 12-case matched-treatment denominator.

E2 supplied CSA and CodeQL detectors from an independent generator on ten patches. Its historical 20-row table mixes automatic and manual or skipped artifacts, so it establishes interface feasibility only, not cross-generator effectiveness; immutable generator/checker lineage remains required.

Historical E3 selected five cases and one decode per cell; it is exploratory. The new full-12-case E3 uses three decodes per arm under matched checker, model, validation scope, and per-case call ceiling. These are repeated decodes on the *same* 12 subjects, not 36 independent cases.

There is no separate LLM mechanism classifier to score: the available preflight maps patch/analysis facts to prioritized requirements using fixed rules, while the editor can request roles in the general workflow. The strict CSA comparison freezes the internal view and disables such requests. We therefore do not report classifier accuracy or claim that the fixed rule priorities are optimal. A routing-sensitivity study would hold checker, model, and

Table 3. Evaluation terms and acceptance criteria.

Term	Definition	Success?	Why
Buggy hit	checker reports the target on the vulnerable version	needed	target preservation
Fixed silent	no target alert on the fixed version	not alone	may over-prune
PDS	buggy hit $\wedge$ fixed silent	yes	semantic discrimination
Validity loss	fixed silent but vulnerable hit lost	no	over-pruning
Fixed-noisy	buggy hit but fixed side not silent	no	target failure mode

budget fixed while perturbing visible roles; the historical five-case ablation is not such a study.

5.3 Metrics

The primary metric is *patch-discriminating success* (PDS): a detector must report the target behavior on the vulnerable revision and emit no target warning on the fixed revision. We write PDS over a detector D , patch P , and validation scope V :

$$\begin{aligned}
 H_v(D, P, V) &= \mathbb{K}[\text{alerts}(D, V_{\text{vuln}}, P) > 0], \\
 S_f(D, P, V) &= \mathbb{K}[\text{alerts}(D, V_{\text{fix}}, P) = 0], \\
 \text{PDS}(D, P, V) &= H_v(D, P, V) \wedge S_f(D, P, V).
 \end{aligned} \tag{2}$$

We report both components: *buggy hit* for target preservation on the vulnerable side, and *fixed silent* for silence under the same fixed-side scope. A detector that becomes fixed-side silent by losing the vulnerable-side target is counted as validity loss, not success. Table 3 summarizes the evaluation terms and acceptance criteria.

The refinement gate accepts a candidate D' only when it changes the baseline detector, remains executable for the backend, and satisfies PDS:

$$\begin{aligned}
 \text{Accept}(D', D) &= \text{Changed}(D', D) \wedge \text{AnalyzerOK}(D') \\
 &\quad \wedge \text{PDS}(D', P, V).
 \end{aligned} \tag{3}$$

All conditions use the same PDS definition and track vulnerable-side loss. E1 counts come from object-level **scan-build** logs over patch-related build objects. We separately report a static artifact-review finding as a checker-quality dimension. A compilable candidate is still scanned on both revisions even if review flags its mechanism: review failure does not turn an executable PDS observation into missing data. We report both behavior-only PDS and the subset that also passes review.

For the full 39-checker portfolio, we additionally binarize each checker/revision pair as a target hit. Each vulnerable revision is one positive and its fixed revision one negative; thus precision, recall, and $F_1 = 2TP/(2TP + FP + FN)$ have a denominator of 78 *patch-version decisions*, not individual analyzer alerts or unseen vulnerability sites. We report PDS and fixed-side alert burden alongside this secondary measure because repeated alerts within an object are not independent examples. Failed analyzer executions are unscored rather than converted into false negatives.

5.4 Experimental Environment

The historical experiments ran on an x86-64 machine with an Intel Core i7-14700K CPU exposed as 16 logical CPUs and 15 GiB RAM, using Python 3.10.12. The later matched

reruns use Python 3.12.3 in the pinned Clang 18 container; both arms of each rerun use the same interpreter and toolchain. CSA uses Clang/LLVM 18.1.8 and `scan-build`. CodeQL uses CodeQL 2.23.5 for C/C++. Linux experiments use Linux v6.13 with `allyesconfig` for Clang/LLD 18.1.8. We keep the same patch-local validation scope for each baseline/refined pair and record manifests, logs, evidence bundles, and final reports.

Use of generative AI in the research workflow. LLMs generated starting checkers in KNighter/E2 and edit checkers in SEMWEAVER; evidence requirements and priorities are deterministic. An AI coding assistant (Codex) also assisted with experiment-driver implementation, audit/analysis scripts, and manuscript drafting. It supplied no ground-truth PDS labels: pinned source, compilation, paired analyzer runs, and the target-preserving rule determine reported outcomes. Manual completions are separate. The artifact retains model settings, prompts, exchanges, candidate code, and script digests; a floating model alias is not an immutable revision. AI-assisted research code and interpretations remain subject to author verification before submission.

5.5 Baselines and Variants

The comparison holds the starting detector fixed: the unmodified checker measures reduction, and KNighter’s actual output-feedback loop is the closest prior refinement baseline. Only the within-SEMWEAVER native/no-internal contrast probes the added evidence. Both methods use the same 12 starting checkers, model, patch/revision scope, per-case call ceiling, and independent postvalidation. The 39-checker portfolio retains already-discriminating checkers under PDS-only adoption; historical follow-up outcomes remain separate.

The unmodified detector is the before/after reference (KNighter-derived CSA in E1, independent-generator CSA/CodeQL in E2). E1 also runs KNighter’s own refinement loop; manual completions never enter automatic results.

Historical E3 named three evidence levels: full bundle, no-analyzer-native, and validation-only. However, a retrospective strict-origin audit found no genuinely internal records in those original E1 bundles, so its five-case contrast cannot isolate internal evidence. A new matched ablation uses the provenance-audited bundles: the native arm receives hash-bound Clang CFG/call-graph records, while the no-internal arm receives no such records but keeps the same checker, patch, source context, feedback, model, call cap, and validator. We do not use the historical five-case numbers as proof of native-evidence contribution.

Historical runs used other models and are not pooled. The matched comparison uses GPT-5.6-terra at high reasoning effort in all three arms. The old 19-call one-pass and unequal-budget conditions remain development diagnostics.

5.6 Protocol

E1 uses `scan-build` counts on patch-related objects. We run KNighter’s actual report-triage, repair, compiler-repair, and validation loop first. In each repeat, both SEMWEAVER arms inherit its *actual per-case model-call count* as a frozen ceiling. SEMWEAVER makes at most one model call before each independent compile and paired replay, feeding hash-bound failures into a next permitted call. The arms differ only in verified internal CFG/call-graph availability; successes stop early. Provider interruptions are excluded, not checker failures; none entered the completed repeats. E2 and historical five-case studies remain separate. The artifact retains English templates, frozen inputs, rendered exchanges, analyzer outputs, provenance, and decisions.

Table 4. Full 12-case matched repeats. Each arm cell is PDS / retained fixed-side alerts (starting from 37) / actual model calls. Both SEMWEAVER arms share the same per-case call ceiling and feedback loop.

Repeat	KNightter	Native SEMWEAVER	No-internal SEMWEAVER
1	3/28/25	6/23/17	3/26/16
2	2/27/26	5/23/21	3/28/20
3	6/22/29	7/18/16	3/26/23

6 RESULTS

6.1 RQ1: KNightter-Derived Checkers

Within the 12 selected fixed-noisy cases, every starting checker hits both paired revisions. The original follow-up records five *automatically* accepted PDS edits, six PDS edits requiring manual completion, and one automatic validity loss. Thus, the often-quoted 11/12 is an *assisted* outcome, not the automatic success rate; likewise, 0/12 is the entry criterion, not a KNightter-refinement result. Under an adopt-only-on-PDS policy, the five historical automatic edits retain the vulnerable-side reports and reduce fixed-side object alerts from 37 to 23 in this stratum; the six manual edits are excluded from that reduction. On the full 39-checker portfolio, the 27 initially discriminating checkers remain unchanged, yielding 32/39 PDS after adopting those five edits. At the patch-version decision level, the unmodified portfolio has $TP = 39$, $FP = 12$, $FN = 0$ ($F_1 = 0.867$); the historical adopt-only portfolio has $TP = 39$, $FP = 7$, $FN = 0$ ($F_1 = 0.918$). These are within-patch decisions, not alert-level precision or unseen-code generalization. Crucially, these historical edits were generated before the strict native-evidence audit and cannot demonstrate that analyzer-internal evidence caused their gain.

Table 4 reports three complete matched repeats on the same 12 selected checkers. The cells give PDS count, retained fixed-side alerts under PDS-only adoption (out of 37 initially), and actual model calls, respectively. The call ceiling is matched *per case* to that repeat's actual KNightter calls (25, 26, or 29 total); SEMWEAVER may stop early. KNightter's actual loop reaches 3, 2, and 6 PDS, while native-evidence SEMWEAVER reaches 6, 5, and 7. It retains 23, 23, and 18 fixed-side alerts versus KNightter's 28, 27, and 22: fewer observed false alerts in all three repeats while retaining vulnerable-side hits for every adopted edit. At the 39-checker patch-version decision level, the corresponding hypothetical PDS-only F_1 is 0.929/0.918/0.940 for SEMWEAVER and 0.897/0.886/0.929 for KNightter. These are within-patch decisions, not alert-level or deployment precision.

The exact paired McNemar p -values against KNightter are 0.375, 0.375, and 1.0 by repeat; no single repeat establishes significant superiority. The original 19-call one-pass tie and an unequal-budget development condition remain diagnostics, not part of Table 4. A separate review heuristic accepted all 6, 5, and 7 native PDS candidates, but only 3, 1, and 5 of KNightter's PDS candidates; its findings are quality warnings, not grounds to erase valid paired executions. The three repeats characterize stochastic variability on the same development-informed stratum, not an independent 36-checker cohort.

6.2 RQ2: Cross-Backend Feasibility and Provenance

The historical 20-detector CSA/CodeQL table exercises both formats but mixes at least seven manual-path rows and cannot support an automatic cross-backend gain. Its former

8/20 $\rightarrow$ 12/20 summary is not independently reproducible; E2 remains interface feasibility until generator prompts, checker hashes, manual status, and paired logs are bound per row.

6.3 RQ3: Evidence Sensitivity and Provenance

The matched no-internal arm keeps patch, source, model, feedback, and call ceilings, but lacks hash-bound CFG/call-graph records. It reaches 3/12 PDS in each repeat versus native 6/12, 5/12, and 7/12, retaining 26, 28, and 26 fixed alerts versus 23, 23, and 18. Within-repeat paired p -values are 0.25, 0.625, and 0.125; none is significant alone. Case 02 succeeds only without internal evidence in repeat 2, ruling out a necessity claim. The older purposive five-case ablation had no strictly internal records and remains only a failure-mode illustration.

6.4 Qualitative Findings

The case studies cover resource-lifecycle, initialization, and integer/bounds edits, but patch-local acceptance alone does not show that the resulting checker will recognize unseen sites.

The negative case is equally important: a refinement can silence the fixed side by making the checker too narrow. That is not success, even though the fixed side is silent. Feedback alone cannot tell a correct semantic barrier from an over-pruned detector. In a post-hoc, development-informed case-19 probe of a native PDS candidate, inserting an unrelated statement and renaming the enclosing function preserve the alert, but consistently renaming the cleanup callback loses it; two negative controls remain silent. This 3/4 positive, 2/2 negative result is a scoped robustness diagnostic, not held-out generalization or an additional automatic PDS success.

7 DISCUSSION

Scope of improvement. SEMWEAVER targets patch-related regression checking, not whole-project bug finding or vulnerability-class discovery. A checker may know *where* to warn yet miss *why* the warning should disappear after the fix. The refinement target is a change that reduces fixed-side noise while keeping the vulnerable trigger intact on the paired patch-related objects. Use on a changed fork or later version requires additional stability tests; paired PDS alone does not justify that deployment.

Why post-synthesis and LLM-generated checkers? The patch, paired revisions, and an existing checker give a fixed artifact and an observable fixed-noisy failure to repair. Analyzer evidence might also inform a fresh generator or a handwritten checker, but those settings change the starting artifact or authoring workflow and have not been evaluated here. We separate generation from refinement to attribute any patch-local false-positive reduction to an edit of the same checker, not to a new generation draw; the method is not inherently limited to LLM authorship if an existing checker and paired validation target are supplied.

Evidence as context compression. The relevant semantic condition may span multiple files and analyzer outputs. Passing the full repository to the model is infeasible, while passing only the patch is too sparse. SEMWEAVER bridges this gap by collecting a provenance-tagged summary of patch-relevant facts, compact enough for the agent yet rich enough to guide edits. This positions checker refinement closer to specification completion than to report suppression [11, 34, 36, 37, 45].

Backend complementarity. E2 exercises both CSA and CodeQL detector formats, but does not by itself show that every evidence role was populated from an analyzer-internal representation. In the audited CSA cohort, the verified native interface is a CFG/call-graph

dump; source-derived and diagnostic records remain separately labelled. A CodeQL-native coverage claim requires retained database-query provenance.

Failure modes. The dominant failure mode is over-pruning: a candidate suppresses fixed-side warnings by deleting or over-constraining the vulnerable trigger. PDS rejects such cases as validity loss, never success. Other modes include evidence insufficiency (a collector misses the needed fact), backend mismatch (the analyzer cannot expose the required relation), and model-side edit quality (the model cannot turn a correct fact into compilable code). In each case the gate rejects the candidate rather than silently accepting it.

Limited semantic generalization. SEMWEAVER validates patch-local discrimination, not all instances of a vulnerability class [8, 22, 39, 40]. Some edits introduce a structural predicate; others, including the motivating AST-body edit, are tied to a patch function. A post-hoc case-19 probe survives insertion of an unrelated statement and renaming the enclosing function, but misses a consistently renamed cleanup callback. Thus even an automatic PDS candidate may remain callback-specific; this diagnostic is not a held-out transfer result.

Precision and scaling boundary. The strict gate protects vulnerable-side recall while reducing fixed-side alerts within the patch-related validation scope. Across three matched repeats on the 12 selected cases, native SEMWEAVER retains 23, 23, and 18 of 37 starting fixed alerts, below KNIGHTER's paired 28, 27, and 22; all accepted native candidates remain vulnerable-side hits. The frozen CSA evidence replay completed on all 12 cases with four to nine records per bundle (70 total), and portfolio arithmetic retains all 39 checkers under PDS-only adoption without sending whole repositories to the editor. This is bounded, automated workflow scale in the measured cohort, not whole-kernel throughput or broad vulnerability-class generalization.

8 THREATS TO VALIDITY

Internal. An error in evidence collection, LLM editing, or validation could produce a spurious refinement. The paired gate rejects candidates that lose the vulnerable-side trigger or fail to compile, but cannot establish generalization outside its scope. Static artifact review can flag a mechanism-quality defect without negating executable paired behavior; we report its pass count separately. The original E1 follow-up included six manual completions and its bundles do not substantiate strict analyzer-internal evidence, so those outcomes remain separate. Three high-effort repeats estimate decoding variability for E1 and the full-12-case E3 contrast, but reuse the same development-informed subjects; the additional repeats were declared after the first native decode, not pre-registered as fresh-subject confirmation. Historical E4 remains single-run and purposively selected.

Construct. PDS evaluates discrimination between paired revisions under a patch-related-object scope. It does not measure whole-project false-positive rates or vulnerability-class coverage. The secondary F_1 counts 78 paired patch-version decisions, not individual alerts; it therefore must not be interpreted as a deployment precision estimate. Fixed-side silence is enforced over patch-related build objects rather than just the changed line, but a checker can still overfit the patch function.

External. The frozen cohort comprises 39 materialized Linux checkers, with 12 fixed-noisy under the chosen object scope; representativeness is unproven. A separate ten-candidate screen required repaired-file substitution for empty final checkers and found five already PDS, two fixed-noisy, and three target misses. Those two new noisy cases lack the same frozen model-treatment inputs and do not enter the 12-case matched comparison. Historical cross-backend rows have mixed manual provenance. Refined detectors are validated for

patch-local fidelity, not unseen code or future kernel versions. The case-19 metamorphic probe was designed after inspecting a success and is diagnostic, not a held-out robustness estimate.

9 RELATED WORK

Static Analysis and Checker Languages. Static-analysis systems provide the substrate that SEMWEAVER refines against. Path-sensitive analyzers, abstract interpreters, and symbolic engines expose guards, states, paths, and diagnostics reusable as evidence [3, 6, 12, 14] and deploy at industrial scale [4, 7, 42]. Declarative languages offer a complementary approach by querying program facts through relational rules and points-to specifications [2, 5, 20], while semantic patching encodes recurring transformations over systems code [38]. SEMWEAVER introduces no new analyzer or query language. It treats LLM-generated analyzer artifacts as editable targets and uses analyzer-native facts as evidence for semantic completion.

Vulnerability Detection and Patch Evidence. Static vulnerability detection has long used taint and data-flow reasoning [1, 21, 31], graph and AST representations [46, 47], and learning-based token-, graph-, slice-, and line-level models [16, 29, 30, 41], though datasets and evaluations show that semantic robustness remains difficult [8, 15]. Clone- and patch-driven systems use historical fixes as compact witnesses [19, 24, 28]. These motivate our use of patches as specifications, but SEMWEAVER targets a different artifact: an already generated checker whose fixed-side behavior reveals missing semantics.

LLM-Based Code and Security Reasoning. General-purpose code LLMs can synthesize code [10], but security studies caution against trusting generated artifacts without validation [39, 40]. SEMWEAVER uses an LLM to edit a static-analysis artifact, while paired analyzer executions, rather than the model’s explanation, determine acceptance.

Checker Synthesis and Feedback-Guided Repair. Patch-driven checker synthesis, especially KNighter [48], is the closest prior work. LLMs transform historical security fixes into specialized analyzers that compile, execute, and are reused. SEMWEAVER is complementary because it assumes such a detector exists and refines it post-synthesis. A generator’s validation feedback reveals which side a detector fires on but not whether the missing predicate is a guard, barrier, flow, state, lifetime, type filter, or API-usage relation.

The loop also relates to automated program repair, synthesis, and counterexample-guided workflows that edit candidates under compiler, test, verifier, symbolic, or learned feedback [11, 23, 26, 33, 34, 36, 37, 45], and to LLM refinement and agent work on iterative feedback, tool use, and deliberative search [35, 43, 44, 49, 50]. SEMWEAVER differs in artifact and acceptance: the edit target is a detector, not the vulnerable program, and acceptance is not merely “compiles” or “fewer warnings” but the patch-induced before/after property that rejects report suppression when the target bug is lost.

Analyzer Context for LLMs and Alert Triage. Prior work already uses analysis information in LLM workflows. Chapman et al. interleave static-analysis intermediate results and LLM prompts to infer error specifications [9]; IRIS infers taint specifications and performs contextual analysis around CodeQL results [27]. LLM4PFA reasons about interprocedural path feasibility to filter static bug reports [13], while ZeroFalse enriches alerts with flow-sensitive traces before LLM adjudication [18]. We therefore do not claim to be the first to prompt an LLM with analyzer information. The narrower distinction is the edited artifact and gate: SEMWEAVER changes an existing generated checker, and accepts that code change only if paired patch-revision validation retains the vulnerable hit and silences

the fixed side. Even this distinction is useful only if the evidence origin and checker outcome are independently auditable.

10 CONCLUSION

We presented SEMWEAVER, a post-synthesis layer that combines provenance-labelled analyzer evidence with a gate preserving vulnerable-side reports. In three automated, matched repeats on the 12 fixed-noisy checkers of a 39-checker portfolio, it retained 23, 23, and 18 fixed-side alerts versus KNighter’s 28, 27, and 22, while accepting only patch-discriminating edits. This shows bounded batch precision in paired patch-related code, not whole-kernel throughput or detection of future instances of a vulnerability class.

DATA AVAILABILITY

The anonymized repository is at <https://anonymous.4open.science/r/SemWeaver-FD1E/>. Its English replication package contains the implementation, frozen checker and patch inputs, Clang raw-output/provenance bindings, three matched repeats, the separate ten-candidate screen, model exchanges, and an offline verifier. Historical manual and mixed-provenance results are labelled separately. Full reruns require public Linux revisions and the pinned toolchain; the curated package is intended to remain public upon acceptance.

REFERENCES

- [1] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269. <https://doi.org/10.1145/2594291.2594299>
- [2] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-Oriented Queries on Relational Data. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 2:1–2:25. <https://doi.org/10.4230/LIPICs.ECOOP.2016.2>
- [3] Thomas Ball and Sriram K. Rajamani. 2002. The SLAM Project: Debugging System Software via Static Analysis. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1–3. <https://doi.org/10.1145/503272.503274>
- [4] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Commun. ACM* 53, 2 (2010), 66–75. <https://doi.org/10.1145/1646353.1646374>
- [5] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly Declarative Specification of Sophisticated Points-to Analyses. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 243–262. <https://doi.org/10.1145/1640089.1640108>
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 209–224. https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [7] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th NASA Formal Methods Symposium (NFM)*. 3–11. https://doi.org/10.1007/978-3-319-17524-9_1
- [8] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- [9] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. 2024. Interleaving Static Analysis and LLM Prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State of the Art in Program Analysis*. 9–17. <https://doi.org/10.1145/3652588.3663317>

- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). <https://doi.org/10.48550/arXiv.2107.03374>
- [11] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Proceedings of the 12th International Conference on Computer Aided Verification (CAV)*. 154–169. https://doi.org/10.1007/10722167_15
- [12] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTRÉE Analyzer. In *Proceedings of the 14th European Symposium on Programming (ESOP)*. 21–30. https://doi.org/10.1007/978-3-540-31987-0_3
- [13] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. 2025. Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis. *arXiv:2506.10322* <https://arxiv.org/abs/2506.10322>
- [14] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. 57–72. <https://doi.org/10.1145/502034.502041>
- [15] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 508–512. <https://doi.org/10.1145/3379597.3387501>
- [16] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 608–620. <https://doi.org/10.1145/3524842.3528452>
- [17] GitHub. 2026. CodeQL Documentation. <https://codeql.github.com/docs/>. Accessed: Jun. 27, 2026.
- [18] Mohsen Iranmanesh, Sina Moradi Sabet, Sina Marefat, Ali Javidi Ghasr, Allison Wilson, Iman Sharafaldin, and Mohammad A. Tayebi. 2025. ZeroFalse: Improving Precision in Static Analysis with LLMs. *arXiv:2510.02534* <https://arxiv.org/abs/2510.02534>
- [19] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*. 48–62. <https://doi.org/10.1109/SP.2012.13>
- [20] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV)*. 422–430. https://doi.org/10.1007/978-3-319-41540-6_23
- [21] Nenad Jovanovic, Christopher Kruegel, and Engin Kirda. 2006. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 258–263. <https://doi.org/10.1109/SP.2006.29>
- [22] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *Proceedings of the 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2445–2451. <https://doi.org/10.1109/SMC53992.2023.10394237>
- [23] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 802–811. <https://doi.org/10.1109/ICSE.2013.6606626>
- [24] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. 595–614. <https://doi.org/10.1109/SP.2017.62>
- [25] Yevgeny Kliteynik. 2024. net/mlx5: HWS, fixed double free in error flow of definer layout. Linux kernel commit 5aa2184e29081665f915594bc6de9b7fee6e4883. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=5aa2184e29081665f915594bc6de9b7fee6e4883> Accessed: Jul. 1, 2026.
- [26] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. 2012. A Systematic Study of Automated Program Repair: Fixing 55 out of 105 Bugs for \$8 Each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 3–13. <https://doi.org/10.1109/ICSE.2012.6227211>
- [27] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html

- [28] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: An Automated Vulnerability Detection System Based on Code Similarity Analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*. 201–213. <https://doi.org/10.1145/2991079.2991102>
- [29] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258. <https://doi.org/10.1109/TDSC.2021.3051525>
- [30] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Proceedings of the 25th Network and Distributed System Security Symposium (NDSS)*. <https://doi.org/10.14722/ndss.2018.23158>
- [31] Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*. 271–286. https://www.usenix.org/legacy/events/sec05/tech/full_papers/livshits/livshits.pdf
- [32] LLVM Project. 2026. Clang Static Analyzer Documentation. <https://clang-analyzer.llvm.org/>. Accessed: Jun. 27, 2026.
- [33] Fan Long and Martin Rinard. 2015. Staged Program Repair with Condition Synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 166–178. <https://doi.org/10.1145/2786805.2786811>
- [34] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312. <https://doi.org/10.1145/2837614.2837617>
- [35] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf
- [36] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 691–701. <https://doi.org/10.1145/2884781.2884807>
- [37] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: Program Repair via Semantic Analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781. <https://doi.org/10.1109/ICSE.2013.6606623>
- [38] Yoann Padiou, Julia L. Lawall, and Gilles Muller. 2008. Documenting and Automating Collateral Evolutions in Linux Device Drivers. In *Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems*. 247–260. <https://doi.org/10.1145/1352592.1352618>
- [39] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [40] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. <https://doi.org/10.1145/3576915.3623157>
- [41] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 757–762. <https://doi.org/10.1109/ICMLA.2018.00120>
- [42] Caitlin Sadowski, Jeffrey van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tri-corder: Building a Program Analysis Ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*. 598–608. <https://doi.org/10.1109/ICSE.2015.76>
- [43] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68539–68551. https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf

- [44] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [45] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLoS)*. 404–415. <https://doi.org/10.1145/1168857.1168907>
- [46] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*. 590–604. <https://doi.org/10.1109/SP.2014.44>
- [47] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized Vulnerability Extrapolation Using Abstract Syntax Trees. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*. 359–368. <https://doi.org/10.1145/2420950.2421003>
- [48] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 655–669. <https://doi.org/10.1145/3731569.3764827>
- [49] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 11809–11822. https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X